

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO

a.a 2025/2026

Prof. Alessandro Ricci

[Lab Notes]

TASK-ORIENTED PROGRAMMING IN JAVA

STRUCTURING CONCURRENT PROGRAMS INTO TASKS

Come Master non dovrei essere esposto
a come i Task vengono eseguiti

- **Task** concept
 - abstract, discrete, *independent* unit of work
 - *uncoupled* from the notion of thread
- **Division of Labor** pattern (aka divide-et-impera strategy)
 - identify **task boundaries**
 - tasks as independent activities
 - *not depending on state / results / side effects of other tasks*
 - choose a **task execution policy**
 - objectives
 - good throughput
 - good responsiveness
 - grace-full degradation

i Task non devono avere
dipendenze di tipo waiting

non creare Task che, in
un certo punto, si
sospendono aspettando il
risultato di un altro Task
(se viene fatto in questo
mondo, non si sospende
il Task, ma il Thread che
ci sta sotto: dentro il
codice di un Task non
usare meccanismi che
bloccano i Task sotto
(sincronizzazioni pure),
eccezione per le CS)

STRUCTURING CONCURRENT PROGRAMS INTO TASKS

- Outcomes
 - simplify program organization
 - facilitates error recovery by providing natural transaction boundaries
 - promotes concurrency

1

EXECUTING TASKS SEQUENTIALLY

- SingleThreadWebServer example

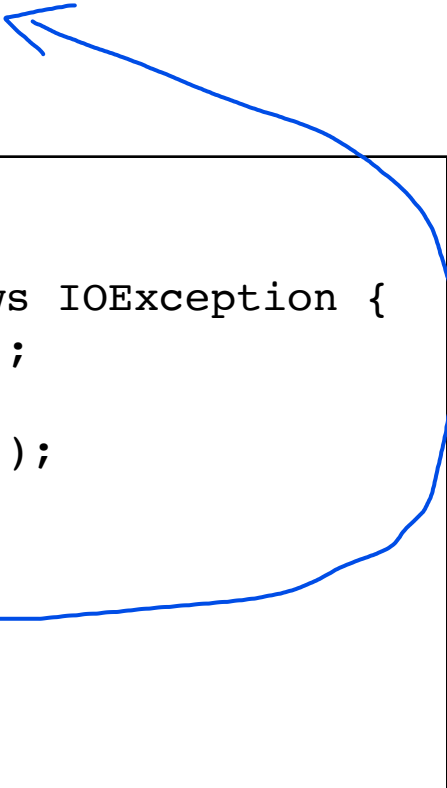
```
class SingleThreadWebServer {  
  
    public static void main(String[] args) throws IOException {  
        ServerSocket socket = new ServerSocket(80); creo server socket  
        while (true) {  
            Socket connection = socket.accept(); Aspetto richieste, le accetto  
ed eseguo la Richiesta  
            handleRequest(connection);  
        }  
    }  
}
```

- Problems
 - poor performance & throughput
 - poor responsiveness
 - poor resource usage

2 ONE-THREAD-PER-TASK POLICY

- Explicitly creating threads for task
 - unbounded thread creation
- ThreadPerTaskWebServer example

Per ogni richiesta, creo
un Thread che la esegue



```
class ThreadPerTaskWebServer {  
  
    public static void main(String[] args) throws IOException {  
        ServerSocket socket = new ServerSocket(80);  
        while (true) {  
            final Socket connection = socket.accept();  
            Runnable task = new Runnable(){  
                public void run(){  
                    handleRequest(connection);  
                }  
            };  
            new Thread(task).start();  
        }  
    }  
}
```

2

ONE-THREAD-PER-TASK POLICY: PROS AND CONS

- Advantages
 - improved responsiveness
 - improved throughput
 - improved resource usage
- Disadvantages & drawbacks
 - thread lifecycle overhead
 - thread creation and teardown are not for free
 - resource consumption
 - active threads consume system resources, especially memory
 - stability
 - there is a limit on how many threads can be created
- Concurrency hazard
 - it may appear to work fine during prototyping and completely fail when deployed under heavy load

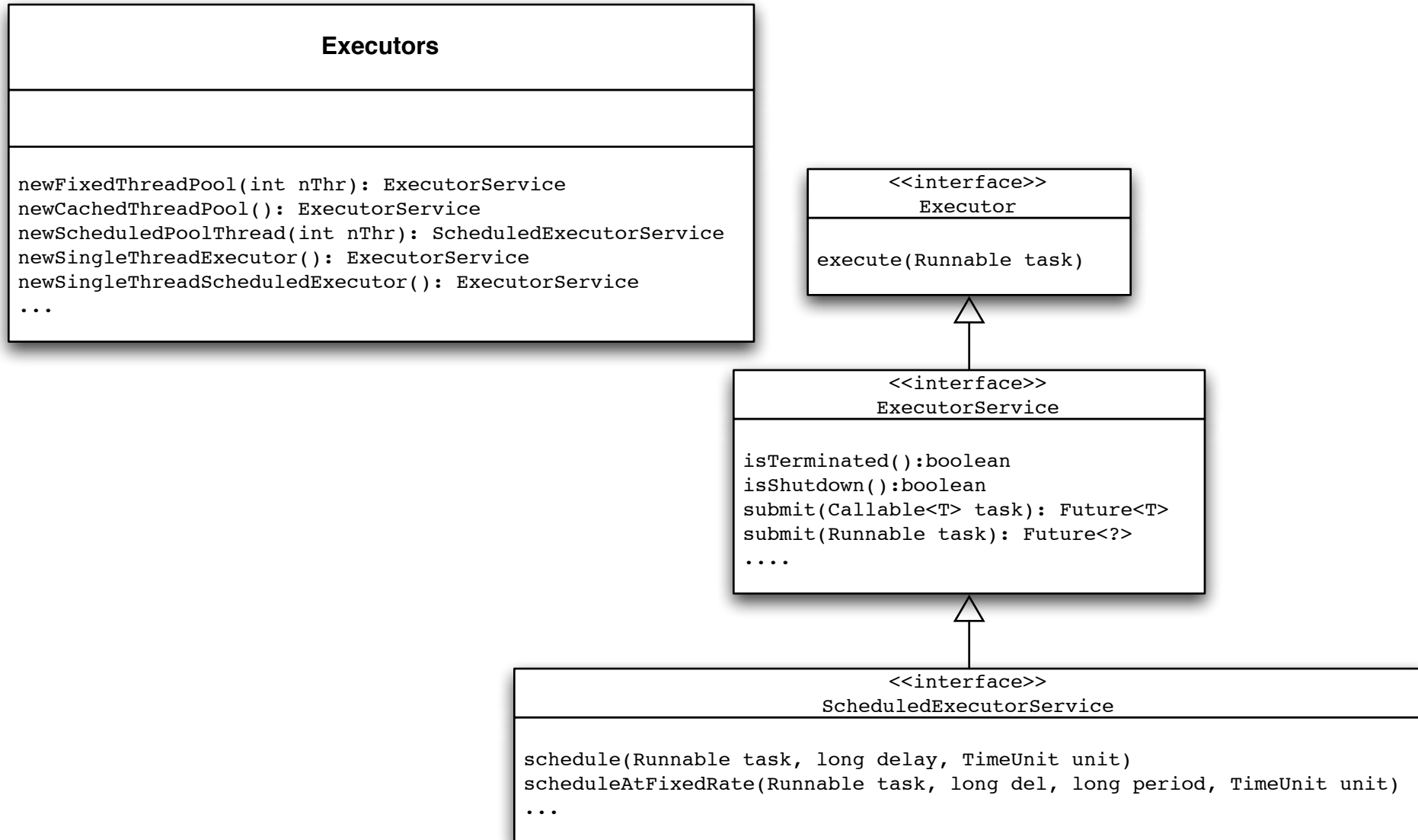
3 MORE FLEXIBLE POLICIES: EXECUTOR FRAMEWORK

- Task-oriented abstraction introduced with JDK 5.0
 - `java.util.concurrent`
- **Direct support for *decoupling task submission* from *task execution***
 - task as logic unit of work
 - threads as the mechanism by which task can run asynchronously
- **Producer-consumer pattern**
 - the producers are the activities that submit tasks
 - the consumers are the threads that execute tasks
- **In the executor framework *task must be independent***
 - *e.g. a task cannot wait for another task*
 - reason: by blocking a task (e.g., on an event semaphore) we are going to block also the physical thread running the task
=> deadlock possibility

FRAMEWORK AND API

- **Executor** Interface
 - execute method to submit tasks
- Task described by **Runnable** interface
 - run method used to define task behavior
- **Executors** class
 - utility for adding statistics gathering, application management and monitoring
 - factory for concrete available executors implementing specific *execution policy* (permette di istanziare tipi di executor già disponibili)

FRAMEWORK AND API



EXECUTION POLICIES

- Specifying the *what, where, when* and *how* of task execution
 - when a task will be executed and by which thread?
 - in what order should tasks be executed (FIFO,LIFO,priority...)?
 - how many tasks may be executed concurrently?
 - If a task has to be rejected because the system is overloaded, which task should be selected as the victim?
 - what actions should be taken before or after executing a task?
- Resource management tool
 - the optimal policy depends on the available computing resources and QoS requirements
- Thread pool management
 - work queue
 - individual thread (consumer) behavior:
 - loop { request next task todo => execute it => go back and wait for another task }

EXECUTORS FACTORY: AVAILABLE EXECUTORS

- per operazioni CPU bound: pool di Thread, con numero di core+1, che fungono da worker per eseguire i task
- per operazioni IO bound, il numero di Thread deve essere il doppio del numero di core disponibili

- **FixedThreadPool**

- one thread for each tasks up to the maximum pool size
- keep the pool size constant

→ Se tutti i thread sono occupati, i nuovi task vengono inseriti in una coda illimitata (unbounded queue) in attesa che un thread si liberi.

- **CachedThreadPool**

Crea nuovi thread al bisogno senza porre limiti superiori (maximumPoolSize = Integer.MAX_VALUE). Riutilizza i thread precedentemente creati se sono liberi; i thread rimasti inattivi (idle) per più di 60 secondi vengono eliminati per liberare risorse.

- reap idle threads when pool size > current demand
- create new threads if needed
- no bounds

- **SingleThreadExecutor**

(un solo thread per eseguire tutto per avere garanzia che ci sia sequenzializzazione)

- single worker, replacing it if dies unexpectedly
- task guaranteed to be processed according to the task queue order (FIFO, LIFO, order)

- **ScheduledThreadPool**

(permette di schedulare dei task e che siano eseguiti ogni tot ms)

- fixed-size thread pool
- support for delayed and periodic task execution

FIRST EXAMPLE: TaskExecutionWebServer

- Using the `FixedThreadPool` policy

```
class TaskExecutionWebServer {  
  
    private static final int NTHREADS  
        = Runtime.getRuntime().availableProcessors()+1;  
    private static final Executor exec =  
        Executors.newFixedThreadPool(NTHREADS);  
  
    public static void main(String[] args) throws IOException {  
        ServerSocket socket = new ServerSocket(80);  
        while (true) {  
            Socket connection = socket.accept();  
            exec.execute(() -> {  
                handleRequest(connection);  
            });  
        }  
    }  
}
```

POOL-BASED VS. THREAD-PER-TASK

- Better performance
- Strong impact on application stability
 - e.g.: a web server no longer fail under heavy load

EXECUTOR SERVICE

- Controlling Executor lifecycle Posso controllare il ciclo di vita degli Executor
 - ExecutorService interface

```
public interface ExecutorService extends Executor {  
  
    void shutdown(); Chiude Executor, per cui i task che sono in esecuzione finiscono, i task che sono sottomessi  
    verranno eseguiti e non si possono dare nuovi task all'Executor (graceful shutdown)  
    List<Runnable> shutdownNow(); Chiude Executor, per cui non accetta più task e non vengono eseguiti quelli  
    sottomessi e si finiscono di eseguire i task in esecuzione (not graceful)  
    boolean isShutdown();  
    boolean isTerminated();  
    boolean awaitTermination(long timeout, TimeUnit unit)  
                                throws InterruptedException;  
    ... permette di sospendere il flusso di controllo chiamante in attesa che  
    l'Executor abbia chiuso tutti i suoi lavori  
    Pattern di utilizzo: chiudo Executor ed aspetto che venga chiuso  
}
```

L'init di un Executor istanzia anche i Thread e la creazione di Thread
è costoso, quindi usare bene lo shutdown

- Defining shutdown policy
 - from graceful shutdown to abrupt shutdown
 - challenging due to task asynchronous behavior

EXECUTOR STATES

- **running**
 - when started
- **shutting down**
 - shutdown method is for *graceful* shutdown
 - no new tasks accepted
 - existing ones are allowed to complete
 - shutdownNow method is for abrupt shutdown
 - it attempts to cancel outstanding tasks
 - does not start any queued (not begun) tasks
 - reject execution handler
 - handling tasks submitted after shutdown
- **terminated**
 - once all tasks have completed

WebServerLifeCycle EXAMPLE

```
class WebServerLifeCycle {  
  
    private static final ExecutorService exec = ...;  
  
    public void start() throws IOException {  
        ServerSocket socket = new ServerSocket(80);  
        while (!exec.isShutdown()) {  
            Socket conn = socket.accept();  
            Runnable task = new Runnable(){  
                public void run(){  
                    handleRequest(conn);  
                }  
            };  
            try {  
                exec.execute(task);  
            } catch (RejectedExecutionException ex){  
                if (!exec.isShutdown()){  
                    log("task submission rejected",ex);  
                }  
            }  
        }  
    }  
    ...  
}
```


WebServerLifeCycle EXAMPLE

```
...

public void stop(){
    exec.shutdown();
}

void handleRequest(Socket connection){
    Request req = readRequest(connection);
    if (isShutdownRequest(req)){
        stop();
    } else {
        dispatchRequest(req);
    }
}
}
```

EXECUTING DELAYED / PERIODIC TASKS

- Working with delayed and periodic tasks
 - scheduling the execution of tasks with absolute and relative timing
- **ScheduledThreadPoolExecutor** class
 - replacement for the Timer class

RESULT-BEARING TASKS: *CALLABLE*

- ExecutorService can manage “callable” tasks
 - tasks representing a deferred computation, completing with some kind of result
 - “call” instead of “run”
- call method encapsulates the behavior of the task doing some kind of computation and returning some kind V of value
- submitted to an executor service by means of submit method, returning a **future**

```
public interface Callable<V> {  
    V call() throws Exception;  
}
```



Mentre l'interfaccia Runnable esegue e basta, l'interfaccia Callable è progettata per riportare informazioni al mittente.

```
public interface ExecutorService extends Executor {  
    ...  
    void execute(Runnable task);  
    Future<T> submit(Callable<T> task);  
    ...  
}
```

Quando il task deve ritornare un risultato, si dà a submit un Callable

FUTURES

- By submitting a task a *future* object is returned
 - provides methods to test whether the task has completed or been cancelled, to retrieve results or cancel the task

```
public interface Future<V> {  
    boolean cancel(boolean mayInterruptIfRunning);  
    boolean isCancelled();  
    boolean isDone();  
    V get() throws InterruptedException, ExecutionException,  
        CancellationException();  
    V get(long timeout, TimeUnit unit) throws InterruptedException,  
        ExecutionException, CancellationException();  
}
```

STOPPING THREADS AND TASKS

- An activity is *cancellable* if external code^(out of the activity code) can move it to completion before its normal completion
- Main reasons:
 - user-requested cancellation
 - e.g.: via GUI
 - time-limited activities
 - e.g. time-bounded search
 - application events
 - e.g. finding the first solution in a searching process
 - errors
 - shutdown
 - e.g. application shutdown

ASYNCHRONOUS VS. COOPERATIVE APPROACH

Come interrompere correttamente un thread? Esistono due modi: uno "brutale" (asincrono) e uno "educato" (cooperativo).

1 • Asynchronous

- **preemption** preemption significa che il sistema toglie forzatamente il controllo a un processo senza il suo consenso.
- **safety problems: state consistency**
 - **API in Thread class deprecated**

Se un thread viene ucciso mentre sta aggiornando una struttura dati (es. sta trasferendo soldi da un conto A a un conto B), potrebbe aver tolto i soldi da A ma non ancora messi in B. I lock (i permessi di accesso) rimangono bloccati o i dati rimangono corrotti. Questi metodi sono deprecati (non vanno usati) perché rendono il software instabile e imprevedibile.

2 • Cooperative (or synchronous) approach

È l'approccio standard moderno. Il thread che vuole fermare l'altro non lo "uccide", ma gli invia una richiesta di stop. Il thread che riceve la richiesta decide quando è il momento sicuro per fermarsi.

- **signaling a request to stop & managing the request**
 - ***cancellation flag*** (Segnale di interruzione): È una variabile booleana (o un flag interno come quello gestito da `thread.interrupt()`) che indica: "Ehi, qualcuno ha chiesto di chiudere tutto".
- **defining the *cancellation policy* for each task** (Politica di cancellazione): Ogni task deve definire la propria strategia. Non è il sistema a decidere, ma lo sviluppatore del task scrivendo il codice.

- **how other code can request cancellation, when the tasks checks whether cancellation has been requested, and what actions the task takes in response to a cancellation request**

Per gestire bene l'interruzione, devi rispondere a tre domande nel tuo codice:

- Come si richiede la cancellazione? Si usa solitamente `future.cancel(true)` o `thread.interrupt()`.
- Quando il task controlla il segnale? Il task non può controllare il flag ogni millisecondo (spreca risorse), ma deve farlo nei punti critici: all'inizio di un ciclo, prima di un'operazione IO pesante o quando entra in uno stato di attesa.
- Cosa fa il task quando riceve la richiesta? Deve pulire le risorse (chiudere file, rilasciare database), loggare l'evento e terminare l'esecuzione in modo pulito (es. con un `return` o lanciando una `InterruptedException`).

CANCELLATION EXAMPLE: PRIME GENERATOR

```
public class PrimeGenerator implements Runnable {  
  
    private final List<BigInteger> primes = new ArrayList<BigInteger>();  
  
    private volatile boolean cancelled;  
  
    public void run(){  
        BigInteger p = BigInteger.ONE;  
        while (!cancelled) {  
            p = p.nextProbablePrime();  
            synchronized (this){  
                primes.add(p);  
            }  
        }  
    }  
  
    public void cancel() { cancelled = true; }  
  
    public synchronized List<BigInteger> get(){  
        return new ArrayList<BigInteger>(primes);  
    }  
}
```

volatile attribute:

to be used when a field can be concurrently accessed by multiple threads.

Enforce the content of the variable to be retrieved and stored in memory each time it is accessed.

CANCELLATION EXAMPLE:

USER SIDE

- Getting one seconds of primes...

```
..  
  
List<BigInteger> aSecondOfPrimes() throws InterruptedException {  
  
    PrimeGenerator gen = new PrimeGenerator();  
  
    new Thread(gen).start();  
  
    try {  
        SECONDS.sleep(1);  
    } finally {  
        gen.cancel();  
    }  
    return gen.get();  
}
```


...AND IF THE THREAD IS BLOCKED?

- Suppose that the bounded buffer is full (blocking put...) cioè il producer dei Prime è bloccato aspettando che il Buffer si liberi

```
class StuckedPrimeProducer extends Thread {  
  
    private BoundedBuffer buffer;  
    private volatile boolean cancelled;  
  
    public StuckedPrimeProducer(BoundedBuffer buf){  
        buffer = buf;  
        cancelled = false;  
    }  
  
    public void run(){  
  
        BigInteger p = BigInteger.ONE;  
  
        while (!cancelled){  
            BigInteger value = p.nextProbablePrime();  
            buf.put(value)  
        }  
    }  
  
    public void cancel(){ cancelled = true; }  
  
}
```

INTERRUPTION

Uno dei vantaggi principali dell'interruzione è che può "svegliare" un thread che è fermo a non fare nulla. Quando un thread sta eseguendo metodi come: sleep(), wait(), join() si trova in uno stato di blocco. Se un altro thread chiama interrupt() su di lui: Il thread bloccato si sveglia immediatamente. Viene lanciata una InterruptedException. Per convenzione, quando viene lanciata questa eccezione, il flag di interruzione viene resettato a false automaticamente.

Agisce sullo stato di interruzione del thread

- **Acting on thread (boolean) interrupted status to avoid infinite blocking**

L'interruzione serve a evitare i blocchi infiniti. Senza questo meccanismo, se un thread andasse in sleep per molto tempo, non avresti modo di chiudere l'applicazione in modo pulito finché non scade il tempo. Grazie a interrupt(), puoi forzare il risveglio e gestire la chiusura.

- set by interrupt() method
- ^{azzerato}cleared by interrupted() method

```
class Thread {  
  
    public void interrupt() {...}  
    public boolean isInterrupted() {...}  
    public static boolean interrupted() {...}  
    ...  
}
```

È il comando di azione. Imposta il flag a true. Se il thread era "addormentato" (in sleep o wait), lo sveglia forzatamente lanciando un'eccezione.

È un metodo di controllo. Restituisce true se il flag è attivo, ma non modifica il flag. Si usa per controllare lo stato senza resettarlo.

È un metodo di controllo e pulizia. Restituisce true se il flag è attivo e lo resetta immediatamente a false. Si usa quando il thread ha gestito l'interruzione e vuole continuare a lavorare pulito.

- **Unblocks blocked states**
 - sleep & wait methods
 - clear the status and throw InterruptedException

INTERRUPTION POLICY

Una Interruption Policy (politica di interruzione) è essenzialmente il manuale di istruzioni di un thread: stabilisce come il thread deve reagire quando riceve un segnale di interruzione. Senza una politica chiara, il rischio è che il thread ignori il segnale (restando attivo all'infinito) o si chiuda in modo brusco lasciando i dati in uno stato incoerente.

- Interruption policy for each thread
 - interruption management idioms

- Common idiom

- explicit shutdown state

Invece di affidarsi solo al flag del thread, si usa spesso una variabile booleana (spesso volatile per garantire la visibilità tra thread) che indica esplicitamente se il task deve terminare. Rende l'intento del codice molto più chiaro e permette di gestire lo spegnimento anche se il thread non è attualmente in uno stato bloccante (come sleep).

- cancel method

Invece di forzare chi chiama il thread a conoscere i dettagli interni (come chiamare `.interrupt()`), si espone un metodo pubblico `cancel()`. Questo metodo incapsula la logica di chiusura: imposta il flag personalizzato e invia l'interruzione ufficiale al thread.

- interruption check in loop

Un task che esegue un lavoro lungo deve essere "educato" e controllare periodicamente se qualcuno ha chiesto di fermarsi. Questo avviene solitamente all'inizio di ogni iterazione del ciclo principale.

- management of InterruptedException

Quando un metodo come `Thread.sleep()` lancia questa eccezione, il flag di interruzione viene resettato. Hai due modi corretti per gestirla:

A. Rilanciare l'eccezione (Propagazione)

Se il tuo metodo fa parte di una libreria o di un task complesso, rilancia l'eccezione per permettere a chi sta sopra di decidere cosa fare.

B. Ripristinare il flag (Preservazione)

Se sei all'interno di un `Runnable` (che non permette di lanciare eccezioni controllate), devi catturare l'eccezione e impostare nuovamente il flag di interruzione.

Nota bene: Se ti limiti a fare un catch vuoto o a stampare solo lo stack trace, "mangi" l'interruzione e il codice chiamante non saprà mai che è stata richiesta la chiusura.

INTERRUPTION POLICY: EXAMPLE

```
class PrimeProducer extends Thread {
    private BlockingQueue<BigInteger> queue;

    public PrimeProducer(BlockingQueue<BigInteger> q){
        queue = q;
    }

    public void run(){
        BigInteger p = BigInteger.ONE;
        try
            while (!Thread.currentThread().isInterrupted()){
                BigInteger value = p.nextProbablePrime();
                queue.put(value)
            } catch (InterruptedException ex){
                // allow thread to exit
            }
        }
    }

    public void cancel(){
        interrupt();
    }
}
```

- Note that calling `interrupt` does not necessary stop the target thread from doing what it is doing
 - it merely delivers the message that interruption has been requested

EXECUTORS SHUTDOWN

Poiché i thread interni sono spesso thread non-daemon, l'applicazione non si chiuderà mai finché l'Executor è attivo. Per questo esistono due strategie di chiusura, che rispecchiano la distinzione tra approccio "educato" e "abrupt".

- Two ways to shut down for ExecutorService

- 1 – **shutdown** method for *graceful* shutdown

L'Executor smette di accettare nuovi compiti, ma si impegna a finire quelli che ha già "in carico".

- no new tasks are accepted
- previously submitted tasks are allowed to complete, including those that have not yet begun execution

Nuovi Tasks: Se provi a chiamare `submit()` dopo aver invocato `shutdown()`, riceverai una `RejectedExecutionException`.

Tasks in corso: Continuano fino alla fine. Tasks in coda: Quelli che erano già stati inviati ma non erano ancora partiti verranno comunque eseguiti.

- 2 – **shutdownNow** method for *abrupt* shutdown

L'Executor cerca di fermare tutto il prima possibile.

- attempts to cancel outstanding tasks
- does not start any tasks that are queued but not begun

Tasks in corso: Viene inviato un `interrupt()` a ogni thread che sta lavorando (qui entra in gioco la `Interruption Policy` di cui abbiamo parlato prima: se il task ignora l'interrupt, non si fermerà!).

- Task submitted to an `ExecutorService` after it has been shutdown are handled by a *rejected execution handler*

Cosa succede se un pezzo di codice invia un task a un `Executor` che è già in fase di spegnimento?
L'Executor delega il problema a un `RejectedExecutionHandler`. Di default, Java lancia una `RejectedExecutionException`, ma puoi personalizzare questo comportamento (ad esempio, loggando l'errore o facendo eseguire il task al thread chiamante per rallentare il flusso).

Tasks in coda: Vengono rimossi dalla coda e mai eseguiti. Valore di ritorno: Il metodo restituisce una `List<Runnable>` contenente tutti i task che erano in coda e che non sono mai stati avviati, permettendoti (se vuoi) di salvarli o loggarli.

Un thread non-daemon (spesso chiamato "user thread") è un thread di esecuzione in un programma che impedisce alla JVM di terminare finché non ha completato il suo lavoro. In altre parole, il programma rimane attivo finché almeno un thread non-daemon è in esecuzione, indipendentemente dal fatto che il thread principale (main) sia terminato.

LifecycleWebServer EXAMPLE

- The web server can be shut down in two ways:
 - 1 – programmatically by calling stop
 - 2 – through a client request by sending a specially formatted request

```
class LifecycleWebServer {
    private final ExecutorService exec = ...

    public void start() throws IOException {
        ServerSocket socket = new ServerSocket(80);
        while (!exec.isShutdown()){
            try {
                Socket sock = conn.accept();
                exec.execute(new Runnable(){
                    public void run(){ handleRequest(conn); }
                });
            } catch (RejectedExecutionException ex){
                if (!exec.isShutdown()){
                    log("task submission rejected.");
                }
            }
        }
    }

    1 public void stop(){ exec.shutdown(); }

    void handleRequest(Socket conn){
        Request req = readRequest();
        2 if (isShutdownRequest(req)) {
            stop();
        } else { dispatchRequest(req); }
    }
}
```

AWAITING FOR TASK COMPLETION

- ExecutorService provides the `awaitTermination` method to await for task completion after a shut down
 - LogService example

```
public class LogService {  
  
    private final ExecutorService exec = Executors.newSingleThreadExecutor();  
    private final PrintWriter writer = ...  
  
    public void stop() throws InterruptedException {  
        try {  
            exec.shutdown();  
            exec.awaitTermination(Long.MAX_VALUE, TimeUnits.SECONDS);  
        } finally {  
            writer.close();  
        }  
    }  
  
    public void log(String msg){  
        try {  
            exec.execute(new WriteTask(msg));  
        } catch (RejectedExecutionException ignored){}  
    }  
}
```

"POISON PILLS" TECHNIQUE

Recognisable objects placed in shared data structures signaling the termination of some process / activity

La tecnica delle Poison Pills è un Design pattern molto elegante utilizzato in particolare nel modello Produttore e Consumatore, per gestire l'arresto ordinato dei thread.

Idea: invece di inviare un segnale esterno per forzare l'arresto, inseriamo un oggetto speciale e riconoscibile direttamente all'interno del buffer, condiviso tra i thread. Quando il consumatore estrae questo oggetto speciale, capisce che è arrivato il momento di fermarsi.

Il flusso di esecuzione si articola nei seguenti punti:

- Il Produttore genera i dati e li inserisce nel BoundedBuffer.
- Quando il produttore deve terminare, inserisce nel buffer un elemento speciale (la Poison Pill), riconoscibile per il suo contenuto (nel tuo esempio, una stringa con descrizione "stop").
- Il Consumatore estrae gli elementi dal buffer uno alla volta.
- Quando il consumatore legge la Poison Pill, riconosce il segnale, imposta la sua variabile di terminazione su true e termina il ciclo.

```
class Consumer extends Thread {  
  
    private BoundedBuffer buffer;  
    private boolean shutdown;  
  
    public Consumer(BoundedBuffer buf){  
        buffer = buf;  
        shutdown = false;  
    }  
  
    public void run(){  
  
        while (!shutdown){  
            try {  
                Item item = buffer.get();  
                if (!item.getDescr().equals("stop")){  
                    // process item  
                } else {  
                    shutdown = true;  
                }  
            } catch (InterruptedException ex){  
            }  
        }  
  
        public void shutdown(){  
            shutdown = true;  
            interrupt();  
        }  
    }  
}
```


INTERRUPTION POLICY: PITFALLS IN JAVA

Sebbene il meccanismo di `interrupt()` sia lo standard in Java, esiste un problema significativo: non tutto è interrompibile. Esistono delle "trappole" (pitfalls) dove un thread può rimanere bloccato per sempre, ignorando qualsiasi chiamata a `interrupt()`.

Le "Trappole": Blocchi non interrompibili

Non-interruptible blocking

- synchronous socket I/O in `java.io`
 - `read / write` to `InputStream / OutputStream`
- (native) lock acquisition
 - `synchronized` blocks

Se un thread è bloccato in un'operazione di lettura o scrittura, su un socket o un file usando le vecchie librerie `java.io`, non risponderà all'interruzione. Il thread rimarrà nello stato `Runnable` (ma bloccato a livello di sistema operativo) finché l'operazione non termina o la connessione non viene chiusa forzatamente.

Quando un thread cerca di entrare in un blocco `synchronized` ed è in attesa che un altro thread rilasci il lock, quell'attesa non è interrompibile. Non puoi forzare un thread a smettere di aspettare un lock intrinseco tramite `interrupt()`.

Le Soluzioni: Rendere tutto interrompibile

Avoiding non-interruptible blocking

- for I/O
 - use `java.nio` library
 - Interruptible channels
 - Selectors are interruptible
- for locking
 - use `java.util.concurrent` library
 - use user-defined mechanisms / media

--> Soluzione per l'I/O: `java.nio` (New I/O)
Le librerie NIO introducono canali e selettori che gestiscono correttamente l'interruzione:
- Interruptible Channels: Se un thread è bloccato su un `InterruptibleChannel` (come un `SocketChannel`) e riceve un `interrupt()`, il canale viene chiuso, il thread riceve una `ClosedByInterruptException` e si sblocca.
- Selectors: Il metodo `select()` (usato per monitorare più canali) è interrompibile: se chiami `interrupt()`, il selettore ritorna immediatamente.

--> Soluzione per i Lock: `java.util.concurrent`
Invece di usare la parola chiave `synchronized`, si utilizzano le classi come `ReentrantLock`. Queste offrono metodi specifici per gestire l'attesa in modo sicuro:
- `lockInterruptibly()`: A differenza del `lock()` normale (o del `synchronized`), questo metodo permette al thread di smettere di aspettare se riceve un segnale di interruzione, lanciando una `InterruptedException`.
- `tryLock(timeout)`: Permette di impostare un tempo massimo di attesa, evitando blocchi infiniti.

MAP-REDUCE ALGORITHMS & THE FORK-JOIN PATTERN (*)

Fork-Join Executor

per gestire meglio i casi in cui ho generazione dei task dinamica (non so a priori quanti task dovrò fare: task creati dinamicamente). Ogni task che faccio può essere ulteriormente suddivisa in sotto-task: il worker che ha preso in carico un task, si accorge che è troppo grande e che lo deve dividere in sotto-task (ricorsivo, fino a che un worker non divide il task ulteriormente): chi fa da worker, può diventare master.

- In parallel programming, many algorithms exist that require tasks to create subtasks and communicate with each other to complete
 - **map and reduce algorithms** (~divide & conquer, master / worker)
 - **map phase** Il problema originale viene diviso in sotto-problemi più piccoli e indipendenti.
 - splitting the data space to be processed by an algorithm into smaller, independent chunks
 - **reduce phase** Una volta che ogni "worker" ha finito di calcolare il risultato del suo piccolo blocco, i risultati parziali vengono sommati tra loro (aggregati) per ottenere il totale finale.
 - once a set of chunks has been processed, partial results can be collected to form the final result
- Executors can be easily applied *when that data space topology is known in advance*
 - simple example: summing elements of an array (see Sums source code)
 - exploiting Callable tasks

(*) Ref: <http://www.oracle.com/technetwork/articles/java/fork-join-422606.html>

Mentre un normale ExecutorService usa una coda di task piatta, il framework Fork-Join è progettato specificamente per task ricorsivi che si sdoppiano.
Fork (Sdoppiamento): Un task lancia un sotto-task per gestire una parte del lavoro. È l'equivalente della fase di Map.
Join (Ricongiungimento): Il task principale aspetta che i sotto-task abbiano finito e ne legge il risultato. È l'equivalente della fase di Reduce.

MAP-REDUCE ALGORITHMS & THE FORK-JOIN PATTERN

Il Problema

If the data space topology is not known in advance (e.g. graph-based and tree-based data structures), *then algorithms should create hierarchies of tasks, waiting for subtasks to complete before returning a partial result*

=> dependencies among sub-tasks

Questa variante (fork-join exec) per evitare il problema riguardante la dipendenza tra task nella implementazione standard degli Executor (possibilità di Deadlock)

=> when a Callable waits for the result of another Callable, it is put in a waiting state, thus wasting an opportunity to handle another Callable queued for execution.

La Soluzione

=> **Fork-Join Framework**

- added to the java.util.concurrent package in Java SE 7 to be effective for map/reduce algorithms

Quando lavoriamo con strutture dati dinamiche (come alberi o grafi), non possiamo dividere il lavoro in parti uguali fin dall'inizio. Dobbiamo creare sotto-task man mano che esploriamo la struttura. Se usassimo un normale ExecutorService con dei Callable, succederebbe questo:

- Il Task A (genitore) sottomette il Task B (figlio).
- Il Task A chiama future.get() per aspettare il risultato di Task B.
- Il thread che esegue il Task A entra in uno stato di attesa (bloccato).

Il problema: Quel thread è occupato a "non fare nulla". Se abbiamo molti livelli di gerarchia, potremmo finire tutti i thread a disposizione semplicemente aspettando i figli, portando il sistema al blocco totale (deadlock) o a un enorme spreco di CPU.

Il framework Fork-Join, introdotto in Java 7, è lo strumento costruito apposta per rendere efficiente questa strategia sui processori moderni:

- Fork (corrisponde al Map): Un task padre divide se stesso in due o più task figli (fork()) e li lancia in parallelo.
- Join (corrisponde al Reduce): Il task padre aspetta la fine dei figli (join()), raccoglie i loro risultati e li aggrega.

FORK-JOIN EXECUTOR & TASKS

- New kind of executor: **ForkJoinPool** executor →
 - dedicated to running instances implementing *ForkJoinTask*
- **ForkJoinTask** objects
 - support the creation of subtasks *plus waiting for the subtasks to complete* ^{e l'attesa}
 - the executor is able to *distribuire* ^{Work-stealing algo} tasks among its internal threads pool by “stealing” jobs when a task is waiting for another task to complete and there are pending tasks to be run ^{spiegazione nella slide successiva}
- **ForkJoinTask API**
 - **fork()** - to launch a new ForkJoinTask from an existing one
 - **join()** - to wait for its completion
 - **invokeAll(Collection<? extends Callable<T>> tasks)** - launch and wait for completion of the specified forkjoin tasks.. → *Sottomette tutti i task specificati e aspetta che siano tutti completati. È preferibile a chiamare manualmente fork() e join() su ogni singolo task perché ottimizza l'ordine di esecuzione.*
- Two types of ForkJoinTask specializations:
 - **RecursiveAction**: executions that do not yield a return value.
 - **RecursiveTask**: yield return values.
- Different synchronous and asynchronous options for the task execution
 - *Sincrona (invoke): Avvia il task e aspetta che l'intera gerarchia sia completata. È il punto di ingresso comune dal codice "normale" al mondo Fork-Join.*
 - *Asincrona (execute o submit): Invia il task al pool e ritorna immediatamente, permettendo al thread chiamante di fare altro.*

A differenza dei normali ThreadPoolExecutor, il ForkJoinPool è progettato per gestire compiti che si aspettano a vicenda.
- Work-Stealing Algorithm: È la caratteristica chiave. Ogni thread nel pool ha una propria coda di task. Se un thread finisce il suo lavoro e la sua coda è vuota, “rubata” (stealing) un task dal fondo della coda di un altro thread impegnato.
- Efficienza nell'attesa: Quando un task chiama join() su un sotto-task, il thread non si mette in stato di “sleep” (spreca risorse), ma cerca altri task da eseguire per rimanere produttivo.

FORK-JOIN EXAMPLE: WORDS COUNTING

- Counting the words of all the documents in a folder, with recursion
 - <http://www.oracle.com/technetwork/articles/java/fork-join-422606.html>
 - lab material

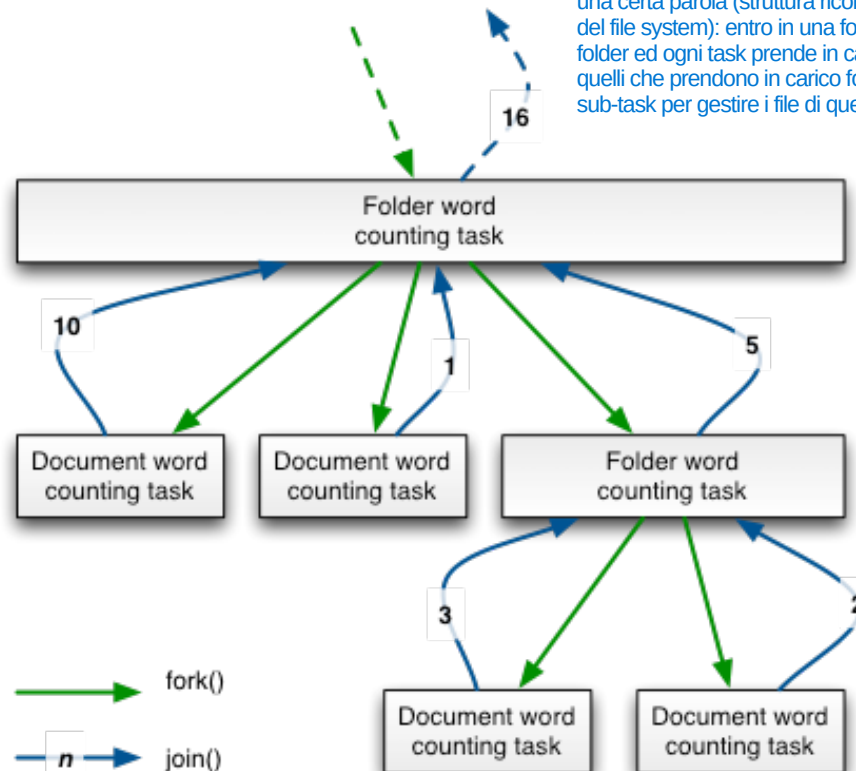
In un Executor normale, esiste una sola grande coda centrale dove tutti i thread pescano il lavoro. Nel ForkJoinPool, invece:

- Ogni thread del pool (chiamato worker thread) possiede la propria coda personale.
- Questa coda è una Deque (Double-Ended Queue), ovvero una coda a due entrate dove si può aggiungere o togliere elementi sia dalla "testa" che dalla "coda".

Quando un thread genera dei sotto-task (tramite `fork()`), li inserisce nella propria coda. Il thread lavora seguendo una logica LIFO (Last-In, First-Out): prende l'ultimo compito che ha appena creato. Perché l'ultimo compito creato è solitamente il più piccolo e i dati necessari per eseguirlo sono probabilmente ancora nella cache della CPU. Questo massimizza le performance.

Il problema sorge quando un thread finisce tutti i suoi compiti o è in attesa (in `join()`) del risultato di un altro task. Invece di restare fermo a guardare: Il thread diventa un "ladro" e guarda nelle code degli altri thread del pool. Se trova un thread che ha ancora molti compiti in coda, ne "rubano" uno. Il furto avviene dal fondo della coda altrui (il compito più vecchio) perché il compito più vecchio è tipicamente quello più grande (il "pezzo" di dati più grosso non ancora suddiviso). Rubando un pezzo grande, il thread ladro potrà poi suddividerlo a sua volta, lavorando per un bel po' senza dover rubare di nuovo.

Analisi di un insieme di file del file system per contare quanti sono i documenti che contengono una certa parola (struttura ricorsiva/gerarchica del file system): entro in una folder che ha altre folder ed ogni task prende in carico folder/file e quelli che prendono in carico folder, creano sub-task per gestire i file di quella sub-folder



STRUCTURED CONCURRENCY: PRELIMINARY VIEW

la Structured Concurrency cerca di risolvere il caos che si crea quando i task asincroni diventano complessi e difficili da monitorare.

- Recent extension to improve task-oriented approaches: **Structured Concurrency**
 - JEP 437 - <https://openjdk.org/jeps/437>
 - delivered with JDK 20 (still in “incubation” state)
- Approach to multithreaded programming *that aims at preserving the readability, maintainability, and observability of single-threaded code*
- Main API design principle

if a task splits into concurrent subtasks then they all return to the same place, namely the task's code block

Il problema principale della concorrenza classica è che i task sono "orfani": se un task principale lancia un thread in background, quel thread può continuare a vivere anche se il padre muore o fallisce. Questo rende il debugging un incubo.

La Structured Concurrency impone una regola ferrea:

Se un task si divide in sotto-task, tutti i flussi devono ricongiungersi nello stesso blocco di codice in cui sono nati.

È esattamente quello che succede con le strutture di controllo classiche come i cicli if o for: entrano in un punto e ne escono in un punto, rendendo il codice leggibile dall'alto verso il basso.

- Readability (Leggibilità): Il codice asincrono sembra codice sequenziale. Non devi gestire complessi callback o catene infinite di `CompletableFuture`.

- Maintainability (Manutenibilità): Poiché i task hanno una gerarchia chiara (padre-figlio), è facile capire chi ha lanciato cosa. Se il padre viene cancellato, i figli vengono cancellati a cascata automaticamente.

- Observability (Osservabilità): I "Thread Dumps" (istantanee dei thread) diventano leggibili. Vedrai una gerarchia di task simile a uno stack trace, rendendo immediato capire dove si è bloccato il sistema.

API: StructuredTaskScope

L'interfaccia `StructuredTaskScope` è lo strumento pratico che traduce la teoria della Structured Concurrency in codice. Il suo obiettivo è eliminare i rischi legati ai thread "orfani" o "zombie", garantendo che la vita di ogni sotto-task sia strettamente legata al blocco di codice che l'ha generato.

- This class allows developers to structure a task as a *family of concurrent subtasks*, and to coordinate them as a unit
 - subtasks are executed in their own (virtual) threads [*] by forking them individually and then joining them as a unit and, possibly, cancelling them as a unit
 - the subtasks' successful results or exceptions are aggregated and handled by the parent task
- Key point
 - *StructuredTaskScope confines the lifetimes of the subtasks, or forks, to a clear lexical scope in which all of a task's interactions with its subtasks — forking, joining, cancelling, handling errors, and composing results — takes place*

Con `StructuredTaskScope`, tutto avviene dentro un blocco `try-with-resources`. Questo crea un recinto (scope) invalicabile:

- Inizio: Creazione dello scope.

- Vita: Fork dei sotto-task.

- Conclusione: Attesa (join) e chiusura automatica.

Se provi a usare un sotto-task al di fuori di questo blocco, il compilatore o il sistema ti fermano. La vita dei thread figli non può superare quella del padre.

[*] Virtual Threads are presented in module-lab-1.8

StructuredTaskScope EXAMPLE

```
// sequential
Response handle() throws IOException {
    String theUser = findUser();
    int    theOrder = fetchOrder();
    return new Response(theUser, theOrder);
}
```

```
// concurrent
Response handle() throws ExecutionException, InterruptedException {
    try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
        Future<String> user = scope.fork(() -> findUser());
        Future<Integer> order = scope.fork(() -> fetchOrder());

        scope.join();           // Join both forks
        scope.throwIfFailed();  // ... and propagate errors

        // Here, both forks have succeeded, so compose their results
        return new Response(user.resultNow(), order.resultNow());
    }
}
```


BIBLIOGRAPHY

- Goetz et al, “Java Concurrency in Practice”, Addison-Wesley
- Julien Ponge, “*Fork and Join: Java Can Excel at Painless Parallel Programming Tool*”
 - <http://www.oracle.com/technetwork/articles/java/fork-join-422606.html>